Assessing Code for Idempotency

It’s extremely hard to assert that a piece of code is idempotent. What you want to do is examine the code triggered from a Sidekiq job and ask yourself what happens when that code fails and retries. A process that helps do this systematically is as follows:

1. For each line of code, N, mentally execute the code from line 1 to N, and then from line 1 to the end of the routine. Don’t worry about *why* the code stops at line N—just assume a gopher chewed through the server’s power cable and everything just stopped.

2. For each N, determine the state of the system after completing the first step.

3. If the state of the system is unacceptable, you’ve identified a likely issue when the Sidekiq job retries after failure.

4. Repeat for all lines of code. Don’t forget to perform this exercise when executing the entire routine twice—failures can happen after your code has run that can trigger a retry!

5. For each unacceptable outcome, rework the code to avoid it using the techniques discussed in this chapter.

This isn’t foolproof, but it provides a relatively repeatable and teachable method to at least think through how the code will behave in the face of failure.